CIS 678 - Machine Learning

– Boosting –

From Weak Learners to AdaBoost

Instructor: Yong Zhuang

yong.zhuang@gvsu.edu

Adapted in part from M.-F. Balcan, “The Boosting Approach to Machine Learning ” CMU 10-401

Last Class

- A decision tree represents a hypothesis for supervised learning.
- Decision trees are learned using a greedy top-down procedure.
- At each node, the algorithm selects a split that improves class purity.
- Common split criteria include:
  - Information Gain
  - Gini Impurity
- Continuous features can be split using threshold values.
- Regression trees predict continuous outputs and commonly use RSS to choose splits.
- Decision trees can become too complex and overfit the training data.
- Pruning and other complexity-control methods can improve generalization.

Agenda

- Ensemble Learning and Boosting
- Weak Learning vs. Strong Learning
- AdaBoost
  - Algorithm
  - Toy Example
- Why Does AdaBoost Work?
  - Training Error
  - Generalization
- Practical Considerations

Boosting

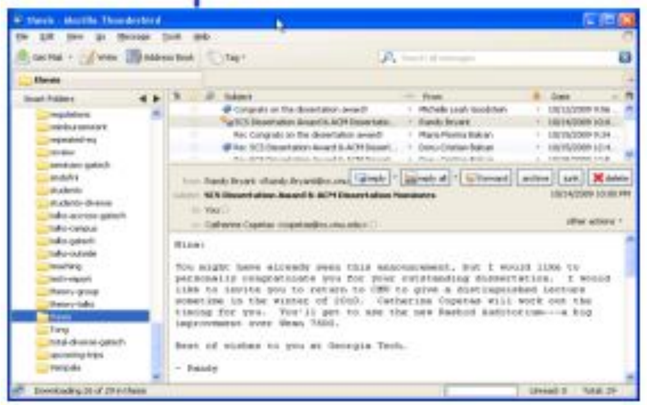
- **Boosting is an ensemble learning method** that combines multiple weak learners to produce a stronger predictor.
- Instead of requiring one highly accurate model, boosting repeatedly trains weak learners.
- Each round focuses more attention on examples that are difficult to classify correctly.
- The weak learners are combined to form the final prediction.

Many weak learners → One strong learner

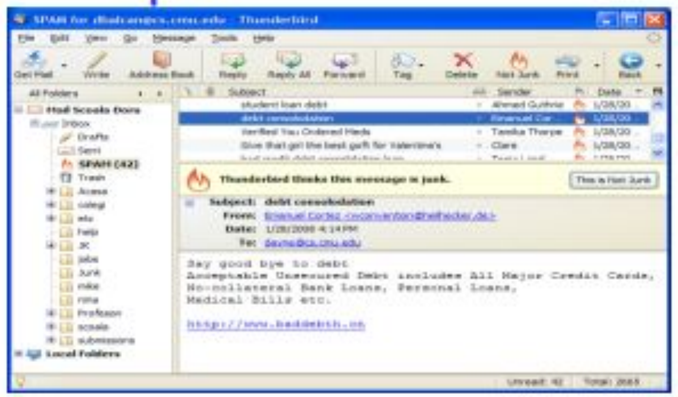
Motivation: Spam Detection

- Goal: classify emails as spam or legitimate

Not spam



spam



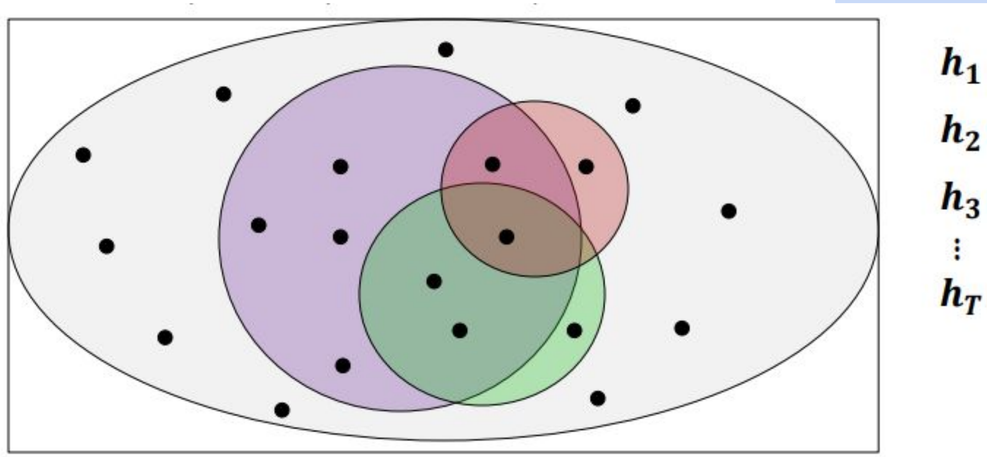
- It is often easy to find simple rules that are somewhat useful:
  - If the message contains **"buy now"**, predict spam.
  - If the message contains **"say good-bye to debt"**, predict spam.
- Each rule is useful, but none is highly accurate by itself.

Question: Can we combine many simple rules into one accurate classifier?

Boosting: Repeatedly Learn from Hard Examples

Boosting is a meta-procedure that repeatedly calls a **weak learner**.

- Start with all training examples equally weighted.
- Train a weak classifier h_1
- Increase the weights of examples that h_1 misclassifies.
- Train another weak classifier h_2 using the updated weights.
- Repeat for T rounds
- Combine the weak classifiers into one final strong classifier.



$$D_1 \rightarrow h_1 \rightarrow D_2 \rightarrow h_2 \rightarrow \cdots \rightarrow D_T \rightarrow h_T$$

Weak learners + reweighting $\rightarrow$ strong classifier

Boosting: Two Key Questions

- **Which examples should we focus on?**
  - Give more attention to hard examples.
  - In AdaBoost, examples misclassified by the current weak learner receive higher weight in the next round.
- **How should we combine the weak learners?**
  - Use a weighted majority vote.
  - More accurate weak learners receive more influence in the final prediction.

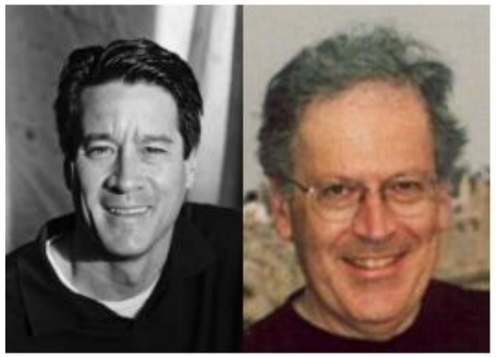
Focus on mistakes + Weighted voting

A Brief History of Boosting

From weak learning to strong learning

Weak Learning vs Strong Learning

- **Weak Learning**
  - A weak learner can consistently find a classifier that performs slightly better than random guessing: $\epsilon \leq 0.5 - \gamma, \quad \gamma > 0$
  - where γ is the learner's **advantage over random guessing**.
- **Strong Learning**
  - A strong learner can produce a classifier with arbitrarily small error: $\epsilon \leq \varepsilon$ for a desired small $\varepsilon>0$.
- **The Boosting Question**
  - Can we transform a weak learner into a strong learner?





Kearns & Valiant (1988): introduced the weak-learning framework and posed the weak-to-strong learning question.

Weak Learning Can Be Boosted to Strong Learning



Schapire (1989) showed that:

- A weak learner can be transformed into a strong learner.
- The boosting procedure runs in polynomial time.
- The weak learner is called repeatedly on different distributions over the training examples.
- The original construction first achieves a modest improvement, then recursively amplifies the accuracy.

Key result:

$$\text{Weak Learning} \Longrightarrow \text{Strong Learning}$$

The original construction was theoretically important, but not very practical.

AdaBoost and Subsequent Work

- **Freund & Schapire (1995)** introduced **AdaBoost**.
- AdaBoost provided a simple and practical way to implement boosting.
- It led to extensive research in:
  - boosting theory,
  - new boosting algorithms,
  - and applications across many domains.

Weak Learning Theory → AdaBoost → Broad ML Applications

Adaboost (Adaptive Boosting)

Freund & Schapire

"A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting"

JCSS'97

Godel Prize winner 2003

Informal Description of AdaBoost

- Boosting: turns a weak algo into a strong learner.

Input: S={(x_1, y_1), …,(x_m, y_m)}; $x_i \in X$, $y_i \in Y = \{-1,1\}$

weak learning algorithm A (e.g., Naïve Bayes, decision stumps)

For t=1,2, … ,T

1. Maintain a distribution D_t on {x_1, …, x_m} over the training examples.
2. Train A using the examples weighted by D_t, producing h_t: $X \rightarrow \{-1,1\}$ (weak classifier)
3. Compute the weighted error $\epsilon_t = P_{x_i \sim D_t}(h_t(x_i) \neq y_i)$ error of h_t over D_t
4. Update the example weights

Output $H_{final}(x) = \text{sign}(\sum_{t=1} \alpha_t h_t(x))$

D_t weights the training examples,
α_t weights the weak classifiers.

Roughly speaking D_{t+1} increases weight on x_i if h_t incorrect on x_i ; decreases it on x_i if h_t correct.

Adaboost (Adaptive Boosting)

- Weak learning algorithm A.
- For t=1,2, … ,T
  - Construct $\mathbf{D}_t$ on $\{\mathbf{x}_1, \ldots, \mathbf{x}_m\}$
  - Run A on D_t producing h_t

Smaller error ϵ_t ⇒ larger α_t ⇒ more influence in the final vote.

Constructing D_t

- D_1 uniform on $\{x_1, \ldots, x_m\}$ [i.e., $D_1(i) = 1/m$]
- Given D_t and h_t set

$$D_{t+1}(i) = \frac{D_t(i)}{Z_t} e^{\{-\alpha_t\}} \text{ if } y_i = h_t(x_i)$$
$$D_{t+1}(i) = \frac{D_t(i)}{Z_t} e^{\{\alpha_t\}} \text{ if } y_i \neq h_t(x_i)$$

$$D_{t+1}(i) = \frac{D_t(i)}{Z_t} e^{\{-\alpha_t y_i h_t(x_i)\}}$$

$$Z_t = \sum_{i=1}^{m} D_t(i) e^{-\alpha_t y_i h_t(x_i)}$$

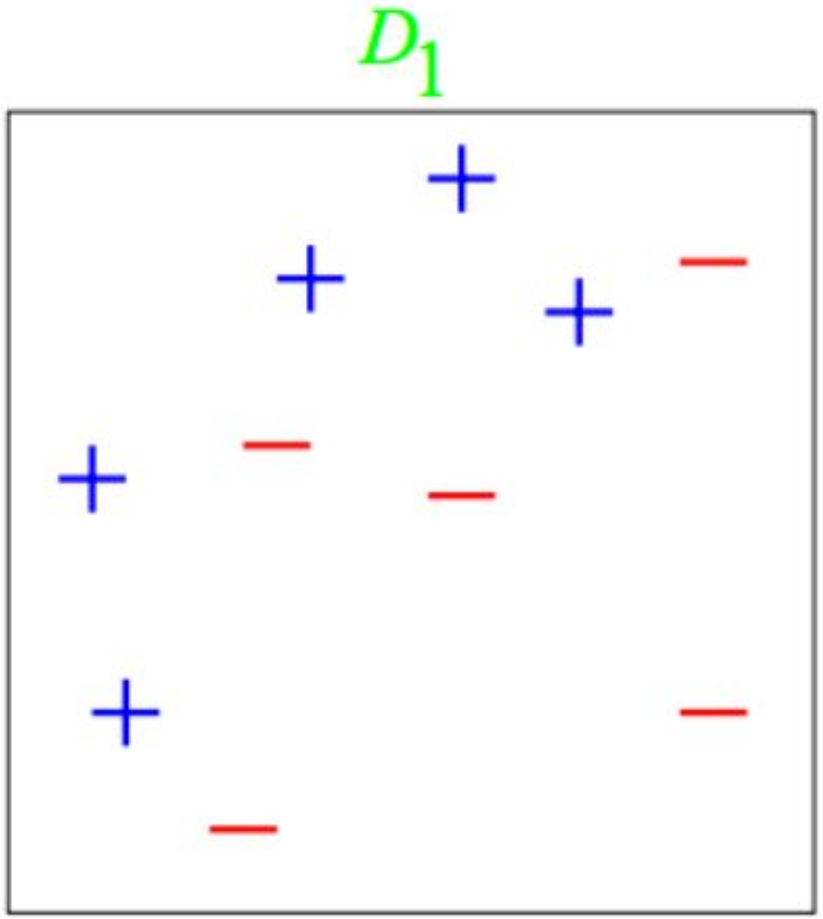
$$\alpha_t = \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right) > 0$$

D_{t+1} puts half of weight on examples where h_t is incorrect & half on examples where h_t is correct

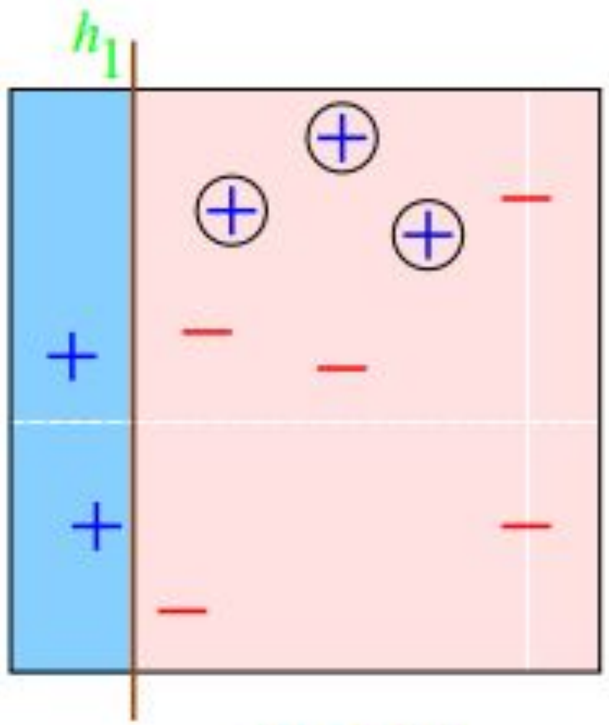
Final hyp: $H_{\text{final}}(x) = \text{sign}(\sum_t \alpha_t h_t(x))$

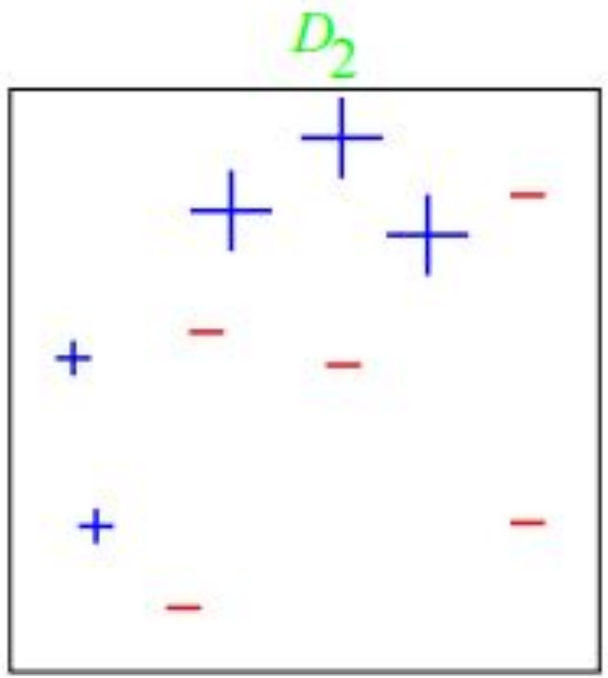
Adaboost: A toy example

- Weak classifiers: vertical or horizontal half-planes (a.k.a. decision stumps)



Adaboost: Round 1



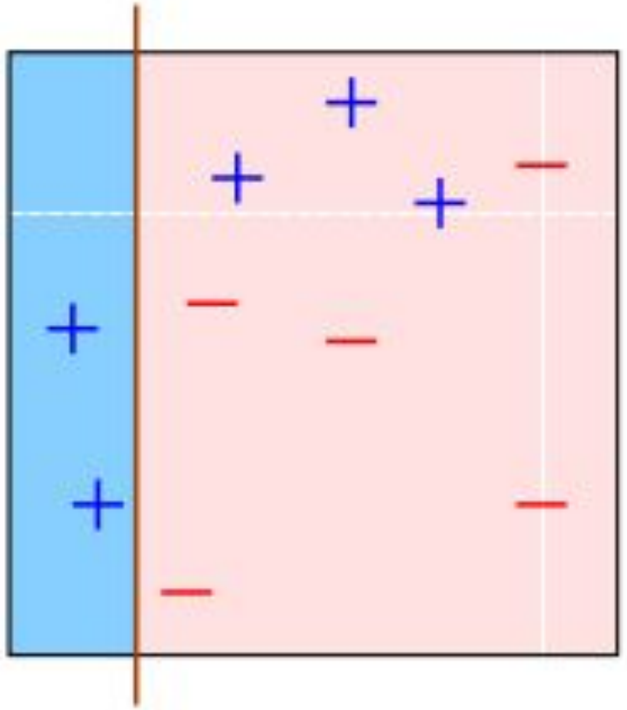


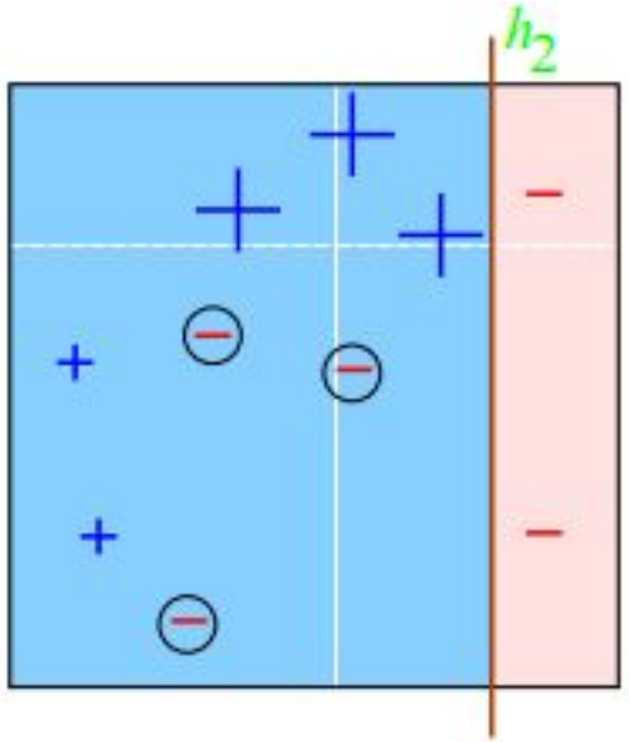
$\varepsilon_1 = 0.30$

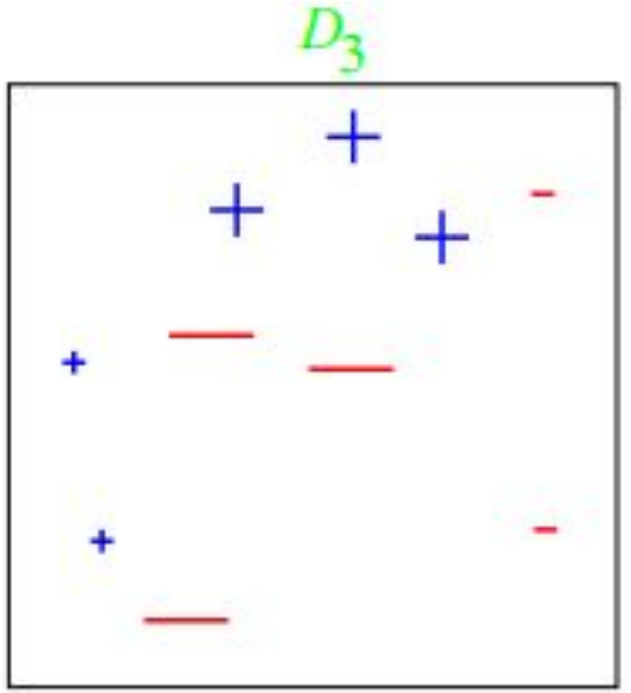
$\alpha_1 = 0.42$

Larger symbols = larger example weights

Adaboost: Round 2







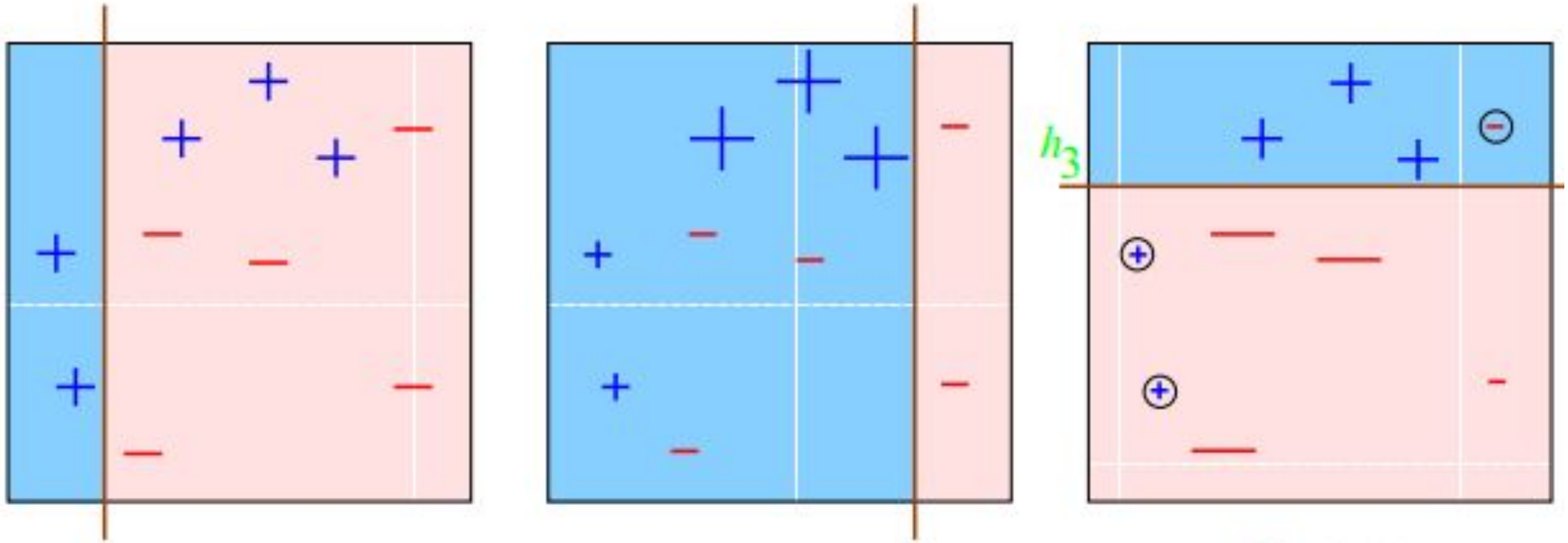
$\varepsilon_2 = 0.21$

$\alpha_2 = 0.65$



$$\epsilon_2 = \sum_{i:h_2(x_i)\neq y_i} D_2(i) = 3\left(\frac{1}{14}\right) = \frac{3}{14} \approx 0.21$$

Adaboost: Round 3



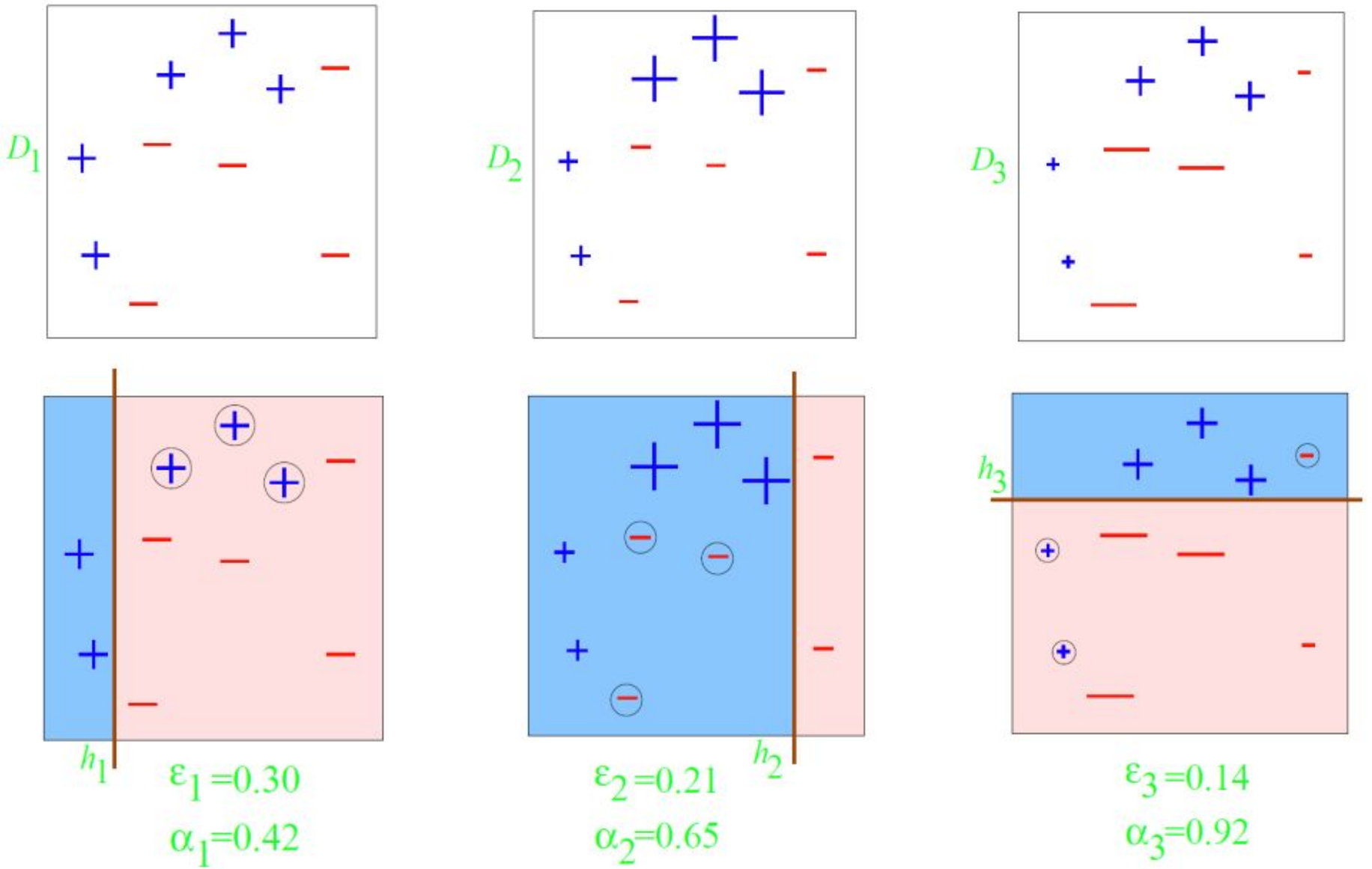
$\varepsilon_3 = 0.14$

$\alpha_3 = 0.92$



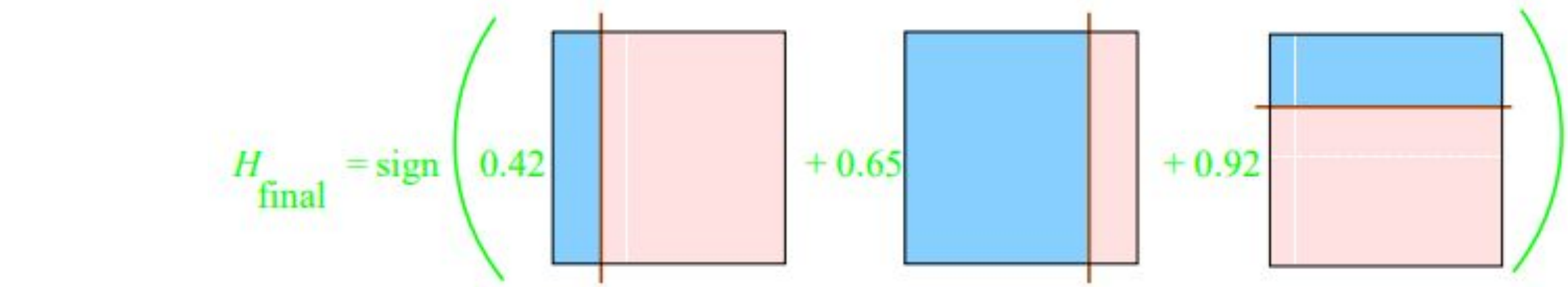
$$\epsilon_3 = \sum_{i:h_3(x_i)\neq y_i} D_3(i) = 0.14$$

AdaBoost: Summary of the First Three Rounds

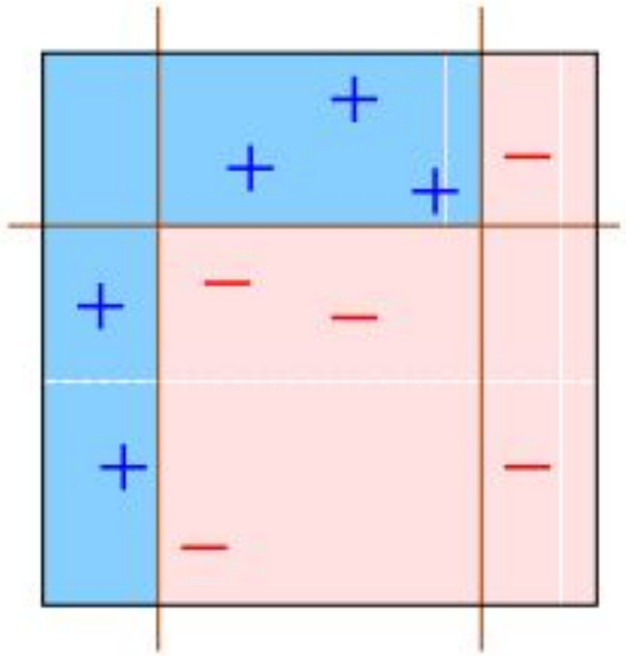


AdaBoost: Final Classifier

$$H_{\text{final}} = \text{sign}\left(0.42 + 0.65 + 0.92\right)$$



$=$



The combination of simple decision stumps can produce a more complex decision boundary.

Adaboost (Adaptive Boosting)

- Weak learning algorithm A.
- For t=1,2, … ,T
  - Construct $\mathbf{D}_t$ on $\{\mathbf{x}_1, \ldots, \mathbf{x}_m\}$
  - Run A on D_t producing h_t

Smaller error ϵ_t ⇒ larger α_t ⇒ more influence in the final vote.

$$Z_t = \sum_{i=1}^{m} D_t(i) e^{-\alpha_t y_i h_t(x_i)}$$

Z_t normalizes the weights so that

$$\sum_i D_{t+1}(i) = 1$$

After normalization, the **total** weight on misclassified examples is 1/2, and the total weight on correctly classified examples is 1/2.

Final hyp: $H_{final}(x) = \text{sign}(\sum_t \alpha_t h_t(x))$

Constructing D_t

- D_1 uniform on $\{x_1, \ldots, x_m\}$ [i.e., $D_1(i) = 1/m$]
- Given D_t and h_t set

$$D_{t+1}(i) = \frac{D_t(i)}{Z_t} e^{\{-\alpha_t\}} \text{ if } y_i = h_t(x_i)$$
$$D_{t+1}(i) = \frac{D_t(i)}{Z_t} e^{\{\alpha_t\}} \text{ if } y_i \neq h_t(x_i)$$

$$D_{t+1}(i) = \frac{D_t(i)}{Z_t} e^{\{-\alpha_t y_i h_t(x_i)\}}$$

$$\alpha_t = \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right) > 0$$

D_{t+1} puts half of weight on examples where h_t is incorrect & half on examples where h_t is correct

Nice Features of Adaboost

- **General**: can be used with many weak learning algorithms.
  - A common choice is a **decision stump**
- **Adaptive :** each round focuses more attention on examples that were previously misclassified.
- **Simple ensemble rule:** weak classifiers are combined using a weighted vote.
- **Powerful idea:** each learner only needs to perform slightly better than random guessing.
- **Strong theoretical foundation:** AdaBoost has well-known guarantees on training error and generalization.

Analyzing Training Error

Theorem ϵ_t = 1/2 − γ_t (error of h_t over D_t)

$$err_S(H_{final}) \leq \exp\left[-2\sum_t \gamma_t^2\right]$$

So, if $\forall t$, $\gamma_t \geq \gamma > 0$, then $err_S\, H_{final} \leq \exp[-2\,\gamma^2 T]$

The training error drops exponentially in T!!!

To get $err_S\, H_{final} \leq \epsilon$, need only $T = O\left(\frac{1}{\gamma^2}\log\left(\frac{1}{\epsilon}\right)\right)$ rounds

Adaboost is adaptive

- Does not need to know γ or T a priori
- Can exploit $\gamma_t \gg \gamma$

Understanding the Updates & Normalization

Claim: D_{t+1} puts half of the weight on x_i where h_t was incorrect and half of the weight on x_i where h_t was correct.

Recall $D_{t+1}(i) = \frac{D_t(i)}{Z_t} e^{\{-\alpha_t y_i h_t(x_i)\}}$

Probabilities are equal!

$$\Pr_{D_{t+1}}[y_i \neq h_t(x_i)] = \sum_{i:y_i \neq h_t(x_i)} \frac{D_t(i)}{Z_t} e^{\alpha_t} = \epsilon_t \frac{1}{Z_t} e^{\alpha_t} = \frac{\epsilon_t}{Z_t} \sqrt{\frac{1-\epsilon_t}{\epsilon_t}} = \frac{\sqrt{\epsilon_t(1-\epsilon_t)}}{Z_t}$$

$$\Pr_{D_{t+1}}[y_i = h_t(x_i)] = \sum_{i:y_i = h_t(x_i)} \frac{D_t(i)}{Z_t} e^{-\alpha_t} = \frac{1-\epsilon_t}{Z_t} e^{-\alpha_t} = \frac{1-\epsilon_t}{Z_t} \sqrt{\frac{\epsilon_t}{1-\epsilon_t}} = \frac{\sqrt{(1-\epsilon_t)\epsilon_t}}{Z_t}$$

$$Z_t = \sum_i D_t(i) e^{-\alpha_t y_i h_t(x_i)} = \sum_{i:y_i = h_t(x_i)} D_t(i) e^{-\alpha_t} + \sum_{i:y_i \neq h_t(x_i)} D_t(i) e^{\alpha_t}$$

$$= (1-\epsilon_t) e^{-\alpha_t} + \epsilon_t e^{\alpha_t} = 2\sqrt{\epsilon_t(1-\epsilon_t)}$$

Training Error Bound: Proof Outline

- **Step 1:** unwrapping recurrence: $D_{T+1}(i) = \frac{1}{m}\left(\frac{\exp(-y_i f(x_i))}{\prod_t Z_t}\right)$

where $f(x_i) = \sum_t \alpha_t h_t(x_i)$. [Unthresholded weighted vote of h_i on x_i]

- **Step 2:** $\mathrm{err}_S(H_{final}) \leq \prod_t Z_t$.

- **Step 3:** $\prod_t Z_t = \prod_t 2\sqrt{\epsilon_t(1-\epsilon_t)} = \prod_t \sqrt{1-4\gamma_t^2} \leq e^{-2\sum_t \gamma_t^2}$

Step 1 — Unroll the Weight Updates

- **Step 1:** unwrapping recurrence: $D_{T+1}(i) = \frac{1}{m}\left(\frac{\exp(-y_i f(x_i))}{\prod_t Z_t}\right)$

where $f(x_i) = \sum_t \alpha_t h_t(x_i)$. [Unthresholded weighted vote of h_i on x_i]

Recall $D_1(i) = \frac{1}{m}$ and $D_{t+1}(i) = D_t(i)\frac{\exp(-y_i \alpha_t h_t(x_i))}{Z_t}$

$$D_{T+1}(i) = \frac{\exp(-y_i \alpha_T h_T(x_i))}{Z_T} \times D_T(i)$$

$$= \frac{\exp(-y_i \alpha_T h_T(x_i))}{Z_T} \times \frac{\exp(-y_i \alpha_{T-1} h_{T-1}(x_i))}{Z_{T-1}} \times D_{T-1}(i)$$

$$\ldots\ldots$$

$$= \frac{\exp(-y_i \alpha_T h_T(x_i))}{Z_T} \times \cdots \times \frac{\exp(-y_i \alpha_1 h_1(x_i))}{Z_1}\frac{1}{m}$$

$$= \frac{1}{m}\frac{\exp(-y_i(\alpha_1 h_1(x_i) + \cdots + \alpha_T h_T(x_T)))}{Z_1 \cdots Z_T} = \frac{1}{m}\frac{\exp(-y_i f(x_i))}{\prod_t Z_t}$$

Step 2 — From 0–1 Loss to Exponential Loss

- **Step 1:** unwrapping recurrence: $D_{T+1}(i) = \frac{1}{m}\left(\frac{\exp(-y_i f(x_i))}{\prod_t Z_t}\right)$

where $f(x_i) = \sum_t \alpha_t h_t(x_i)$. [Unthresholded weighted vote of h_i on x_i]

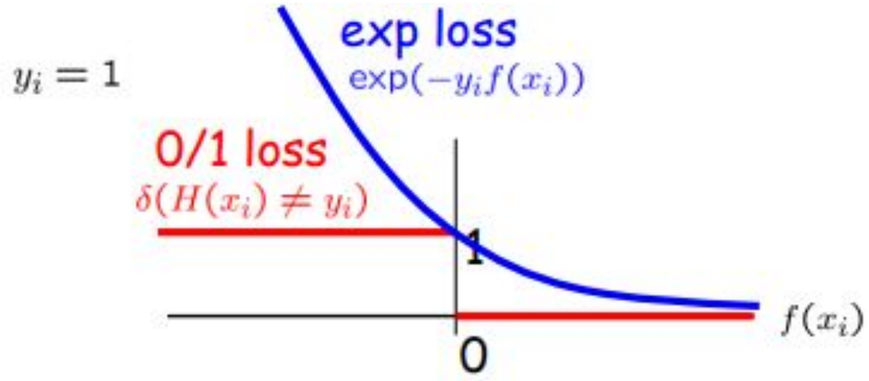
- **Step 2:** $\mathrm{err}_S(H_{final}) \le \prod_t Z_t$.

$$\mathrm{err}_S(H_{final}) = \frac{1}{m}\sum_i 1_{y_i \neq H_{final}(x_i)}$$

$$= \frac{1}{m}\sum_i 1_{y_i f(x_i) \le 0}$$

$$\le \frac{1}{m}\sum_i \exp(-y_i f(x_i))$$

$$= \sum_i D_{T+1}(i) \prod_t Z_t = \prod_t Z_t.$$



Step 3 — Bound the Normalization Constants

- **Step 1:** unwrapping recurrence: $D_{T+1}(i) = \frac{1}{m}\left(\frac{\exp(-y_i f(x_i))}{\prod_t Z_t}\right)$

where $f(x_i) = \sum_t \alpha_t h_t(x_i)$. [Unthresholded weighted vote of h_i on x_i]

- **Step 2:** $\mathrm{err}_S(H_{final}) \leq \prod_t Z_t$.
- **Step 3:** $\prod_t Z_t = \prod_t 2\sqrt{\epsilon_t(1-\epsilon_t)} = \prod_t \sqrt{1-4\gamma_t^2} \leq e^{-2\sum_t \gamma_t^2}$

Recall $Z_t = (1-\epsilon_t)e^{-\alpha_t} + \epsilon_t e^{\alpha_t} = 2\sqrt{\epsilon_t(1-\epsilon_t)}$

α_t minimizer of $\alpha \to (1-\epsilon_t)e^{-\alpha} + \epsilon_t e^{\alpha}$

$$\alpha_t = \frac{1}{2}\ln\frac{1-\epsilon_t}{\epsilon_t}$$

Generalization Guarantees (informal)

Theorem $err_S(H_{final}) \leq \exp\left[-2\sum_t \gamma_t^2\right]$ where $\epsilon_t = 1/2 - \gamma_t$



How about generalization guarantees?

T↑ ⇒ training error decreases

but

T↑ ⇒ complexity term increases

Original analysis [Freund&Schapire'97]

- Let H be the set of rules that the weak learner can use
- Let G be the set of weighted majority rules over T elements of H (i.e., the things that AdaBoost might output)

Theorem [Freund&Schapire'97]

$$\forall\, g \in G, err(g) \leq err_S(g) + \tilde{O}\left(\sqrt{\frac{Td}{m}}\right)$$

T= # of rounds

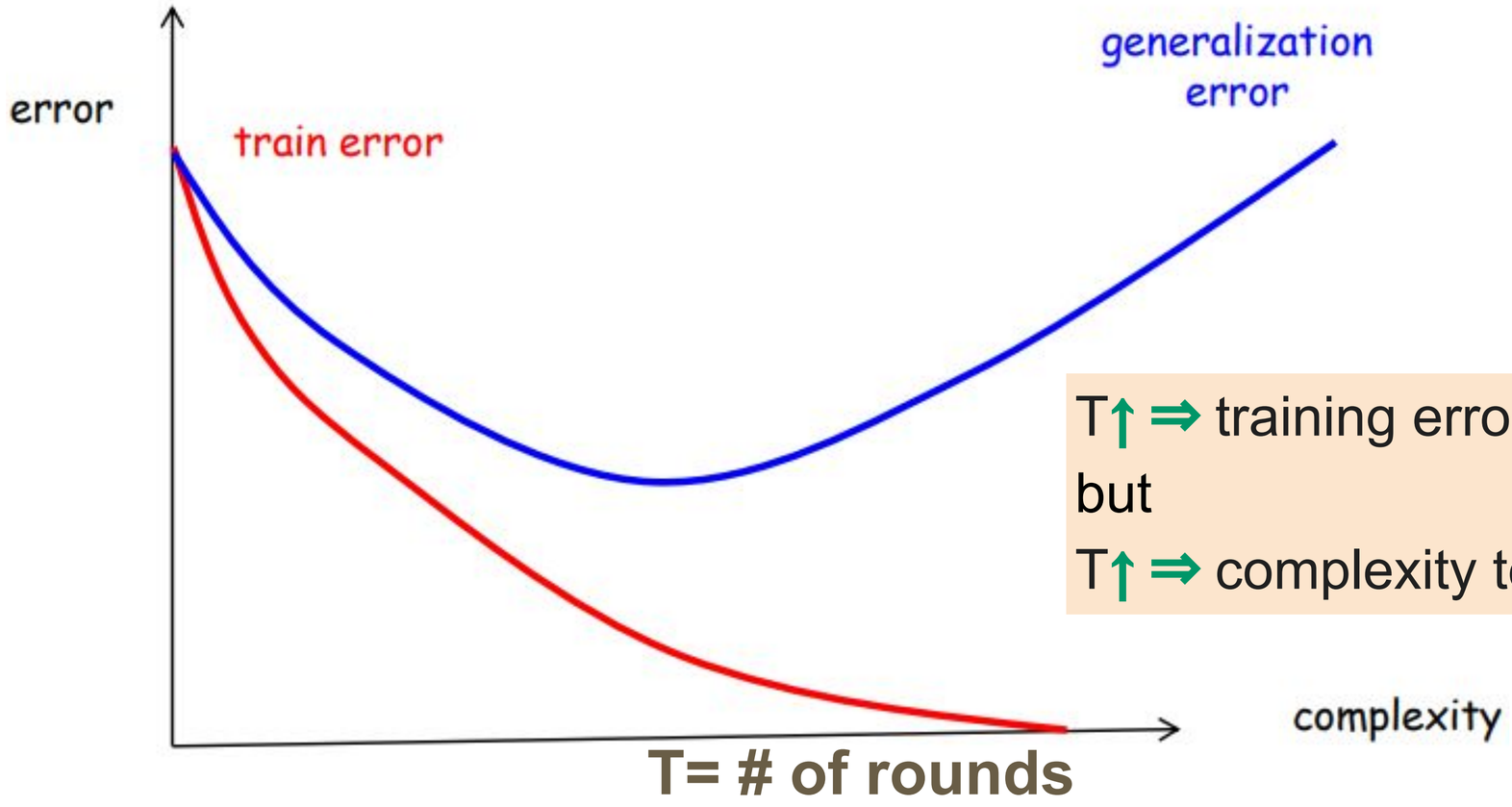
d= VC dimension of H

Generalization Guarantees (informal)

Theorem [Freund&Schapire'97]

$$\forall\, g \in G, err(g) \leq err_S(g) + \tilde{O}\left(\sqrt{\frac{Td}{m}}\right)$$

T= # of rounds
d= VC dimension of H

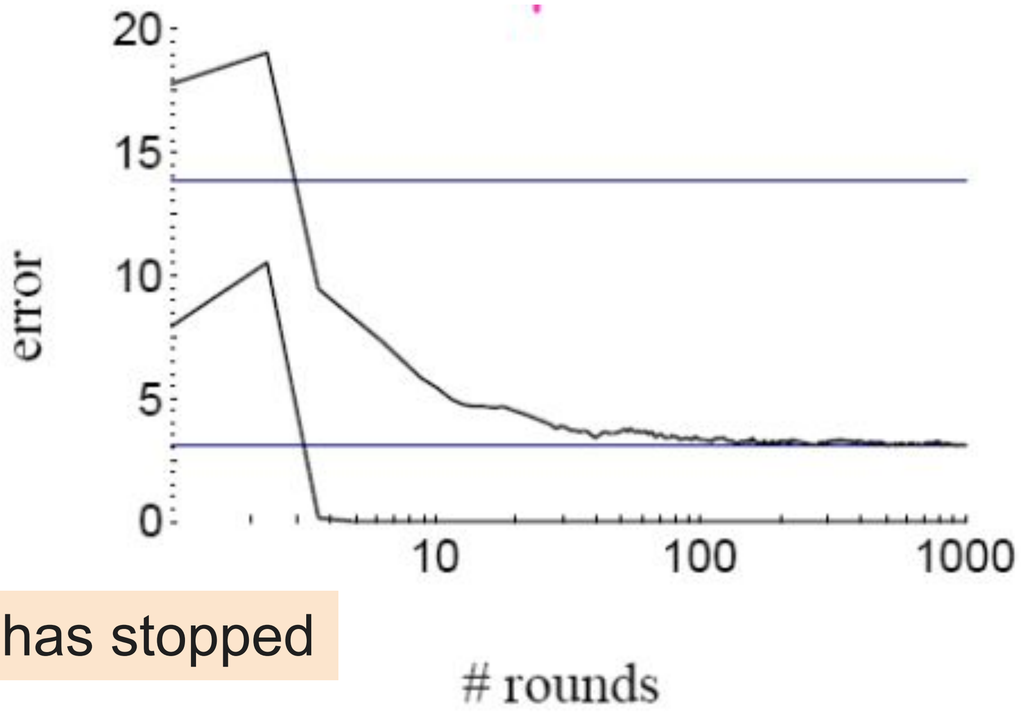


T↑ ⇒ training error decreases
but
T↑ ⇒ complexity term increases

Generalization Guarantees

- Experiments with boosting showed that the test error of the generated classifier usually **does not increase** as its size becomes very large.

- Experiments showed that continuing to add new weak learners after **correct** classification of the training set had been achieved could further **improve** test set performance!!!



Training error = 0 ⇏ learning has stopped

Generalization Guarantees

- Experiments with boosting showed that the test error of the generated classifier usually **does not increase** as its size becomes very large.
- Experiments showed that continuing to add new weak learners after **correct** classification of the training set had been achieved could further **improve** test set performance!!!
- These results seem inconsistent with the simple complexity-based explanation: more rounds make the ensemble larger, yet test error may continue to improve.



Why Can Boosting Keep Improving?

R. Schapire, Y. Freund, P. Bartlett, W. S. Lee. present in “Boosting the margin: A new explanation for the effectiveness of voting methods” a nice theoretical explanation.

Define the ensemble score:

$$F(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$$

For a training example (x_i, y_i), define its margin:

$$\text{margin}_i = y_i F(x_i)$$

- $\text{margin} < 0$: misclassified
- Small positive margin: correct, but low confidence
- Large positive margin: correct with high confidence

Key Idea:

Training error does not tell the whole story.

We need also to consider the classification confidence!!

Summary

- **Boosting** is an ensemble learning approach that combines many weak learners into a stronger model.
- A **weak learner** only needs to perform slightly better than random guessing.
- **AdaBoost** trains weak learners sequentially and focuses more attention on examples that were previously misclassified.
- Each weak learner receives a weight based on its performance, and the final prediction is a **weighted vote** of all weak learners.
- AdaBoost can reduce training error very quickly when each weak learner has a positive advantage over random guessing.
- Increasing the number of boosting rounds increases ensemble complexity, but test error does not necessarily increase.
- **Margins** help explain why boosting may continue to improve generalization even after the training error reaches zero.